Tic-Tac-Toe Architecture

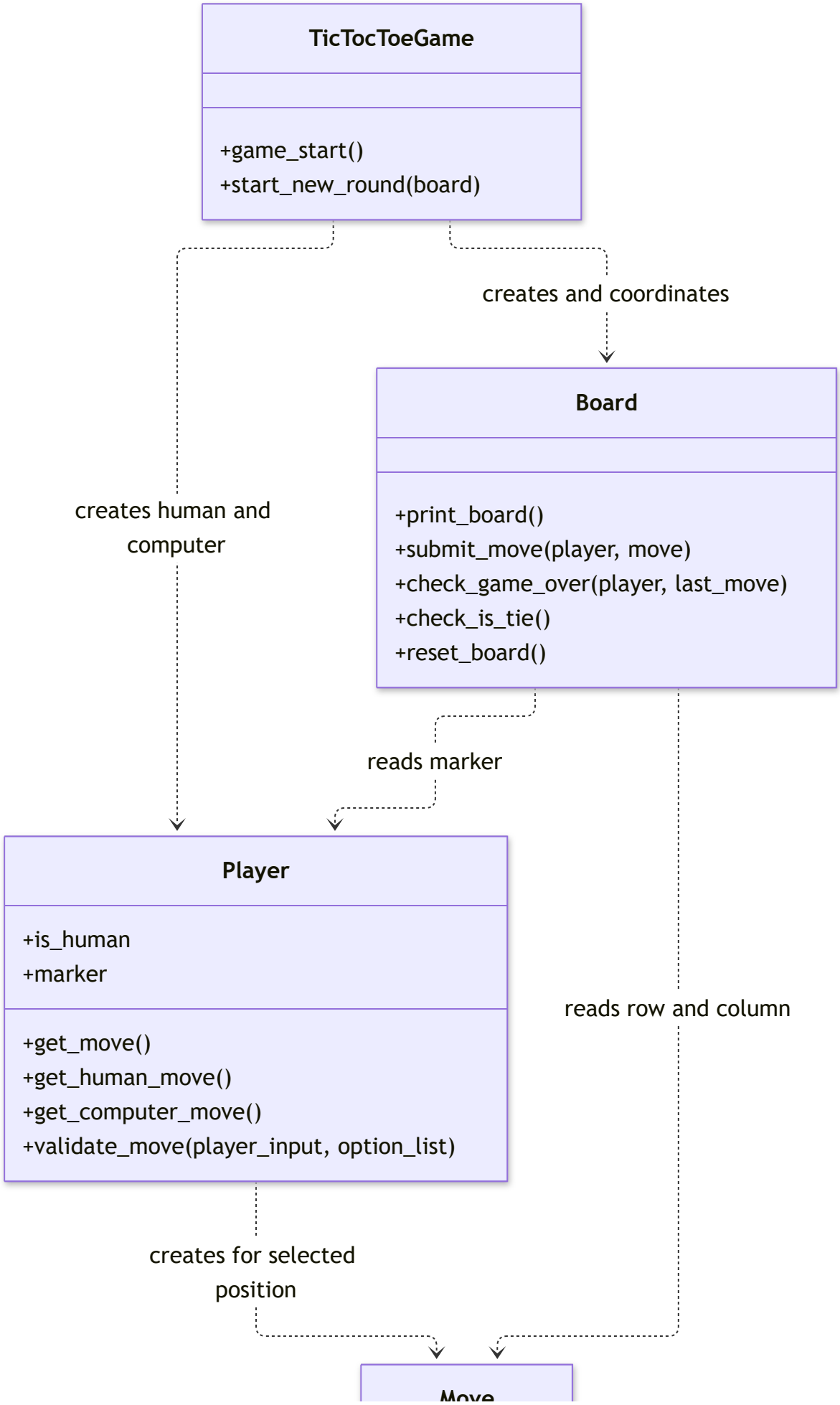
This document describes the command-line human-versus-computer game in `game.py`, `board1.py`, `Player.py`, and `move.py`. It covers the current implementation, including known broken paths, and distinguishes those facts from recommended improvements. It does not describe unrelated examples.

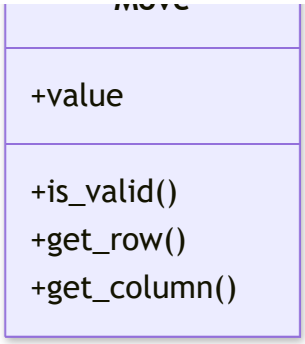
Current implementation

The program is started by running `python game.py`. At module load, `game.py` constructs a `TicTocToeGame` and calls `game_start()`, which creates a `Board`, a human `Player` (`X`), and a computer `Player` (`O`). The game coordinator alternates turns, asks the board to place markers and check results, and prints the board and prompts to the console.

`Move` represents a numbered position from 1 through 9 and maps it to zero-based row and column coordinates. `Player` reads human input or chooses a computer position using Python's `random` module. `Board` owns the 3 × 3 grid, renders it, places markers, and checks for wins and a full-board tie. Board rendering also uses `numpy` and `tabulate` to print the numbered position guide.

Class diagram


TicTocToeGame
+game_start()
+start_new_round(board)
creates and coordinates
creates human and computer
Board
+print_board()
+submit_move(player, move)
+check_game_over(player, last_move)
+check_is_tie()
+reset_board()
reads marker
Player
+is_human
+marker
+get_move()
+get_human_move()
+get_computer_move()
+validate_move(player_input, option_list)
reads row and column
creates for selected position
Move



Human-versus-computer sequence

The diagram shows the implemented turn loop, including the computer-win branch that currently fails to end the round.



High-level architecture flow



Broken execution paths in the current code

- **Computer wins do not end the round.** In `game_start()`, the computer-win branch prints its message but does not break the turn loop. The next human turn can therefore run on an already-won board.
- **Replay raises `AttributeError`.** Choosing `x` resets the board and calls `self.start()`, but `TicTocToeGame` has no `start()` method. The players are also not recreated by `start_new_round()`.
- **Other replay input does not quit.** The invalid-choice branch prints a message but does not break either loop or reset the board; control returns to the outer loop and gameplay continues.

- **Available positions are not persistent or shared.** Each `Player` owns separate lists, while each `get_move()` creates a temporary set from those lists. `validate_move()` removes from that temporary set, not from either player's stored lists or the board. The computer can choose an occupied position again; `Board.submit_move()` only prints a warning and does not request a replacement.
- **Some invalid inputs raise exceptions.** Nonnumeric console input can fail at `int(input(...))` with `ValueError`. In `validate_move()`, a retry choice is removed from the options set before it is checked for membership; an unavailable choice can raise `KeyError`.
- **The program starts on import.** The bottom of `game.py` calls `game_start()` at module scope, so importing that module also starts interactive gameplay.

Recommended improvements (not implemented)

- Make `Board` the single source of truth for legal/occupied positions, and reject invalid or occupied moves before changing game state.
- Validate console input without allowing `ValueError` or `KeyError` to escape; re-prompt for a valid available position.
- End the turn loop immediately on either player's win or on a tie.
- Define one clear replay path that resets the board and player state, and make every non-replay response exit cleanly.
- Put startup behind an explicit entry-point guard so importing `game.py` does not launch the interactive game.
- Add focused tests for move-to-coordinate mapping, occupied positions, win/tie detection, end-of-round control flow, and replay.
- Optionally replace random computer selection with a strategy such as minimax; the current computer is random.